1. System Architecture & Data Flow

```
[GitHub Webhook] → [Node.js Main Service] → [FastAPI AI Agent] → [MCP Server/Tools]
       ↓                       ↓                       ↓
   [Auth/JWT]            [Orchestration]          [Git/Lint/Test/PR Tools]
       ↓                       ↓                       ↓
[PostgreSQL/Prisma] ← [State Tracking] ← [Validation Engine]
       ↓
[WebSocket Server] → [React Frontend (Live Steps)]
```

Core Principles:

- Node.js acts as the orchestrator, auth provider, webhook receiver, and WebSocket broadcaster.
- FastAPI hosts the AI reasoning loop and MCP server for tool execution.
- MCP exposes deterministic, auditable tools (file read/write, git ops, test/lint runners, PR API calls).
- Zero direct main branch writes. All AI changes happen on a temporary branch, validated, then PR'd. Merge only occurs if CI passes + safety gates are green.

. Repository Structures

`main-service/` (Node.js + Express + Socket.IO + Prisma)

```
main-service/
├── src/
│   ├── config/         # DB, JWT, GitHub API configs
│   ├── controllers/    # Auth, PR, Webhook, Status
│   ├── middleware/     # JWT auth, rate-limit, webhook signature
│   ├── routes/         # /api/auth, /api/pr, /api/webhooks
│   ├── services/       # GitHub API, PR orchestrator, WS broadcaster
│   ├── utils/          # Logger, error handler, crypto
│   ├── db/             # Prisma schema, seed, migrations
│   ├── app.js          # Express + Socket.IO setup
│   └── server.js       # Entry point
├── prisma/
│   └── schema.prisma
├── .env
├── package.json
├── tsconfig.json
└── Dockerfile
```

`ai-agent/` (FastAPI + Uvicorn + MCP)

```
ai-agent/
├── app/
│   ├── core/          # Config, logging, LLM client
│   ├── agents/        # PR analyzer, fix planner, validator
│   ├── mcp/           # MCP server definition & tools
│   ├── routes/        # /agent/analyze, /agent/apply, /health
│   ├── utils/         # Git helpers, file ops, diff parser
│   └── main.py        # FastAPI app + Uvicorn entry
├── tests/
├── .env
├── requirements.txt
├── Dockerfile
└── pyproject.toml
```

`frontend/` (React + Vite + Socket.IO)

```
frontend/
├── src/
│   ├── components/    # PRDashboard, LiveLog, StepTracker, Auth
│   ├── hooks/         # useWebSocket, usePRStatus
│   ├── services/      # api.ts, ws.ts
│   ├── store/         # Zustand/Redux for PR state
│   ├── App.tsx
│   └── main.tsx
├── .env
├── package.json
├── vite.config.ts
└── Dockerfile
```

Environment Configurations

- `main-service/.env`

```
NODE_ENV=production
PORT=3000
DATABASE_URL="postgresql://user:pass@localhost:5432/pragent"
JWT_SECRET="your-256-bit-secret-here"
JWT_EXPIRES_IN="24h"
GITHUB_TOKEN="ghp_xxxxxxxxxxxxxxxxxxxx"
GITHUB_WEBHOOK_SECRET="whsec_your_secret"
FASTAPI_URL="http://localhost:8000"
```

`ai-agent/.env`

```
ENV=production
PORT=8000
LLM_API_KEY="sk-xxxxxxxxxxxxxxxx"
LLM_MODEL="gpt-4o" or "claude-3-5-sonnet"
GITHUB_TOKEN="ghp_xxxxxxxxxxxxxxxxxxxx"
NODE_SERVICE_URL="http://localhost:3000"
WORK_DIR="/tmp/ai-agent-workspace"
```

`frontend/.env`

```
VITE_API_URL="http://localhost:3000/api"
VITE_WS_URL="ws://localhost:3000"
```

API Endpoints & Contracts

🌐 4. API Endpoints & Contracts

🔷 Node.js (`main-service`)

Method	Path	Auth	Purpose
`POST`	`/api/auth/register`	❌	Create user, hash password, return JWT
`POST`	`/api/auth/login`	❌	Verify credentials, return JWT + refresh
`POST`	`/api/pr/analyze`	✅	Trigger AI analysis for a PR
`GET`	`/api/pr/:prId/status`	✅	Get current AI run status & steps
`POST`	`/api/webhooks/github`	❌ (HMAC)	Receive `pull_request` events
`WS`	`/ws`	✅ (token in query)	Real-time step streaming

🔷 FastAPI (`ai-agent`)

Method	Path	Auth	Purpose
`POST`	`/agent/analyze`	✅ (Bearer JWT)	Receive PR diff, return analysis + fix plan
`POST`	`/agent/apply-fix`	✅	Apply fixes, run validation, create PR
`POST`	`/agent/webhook`	✅	FastAPI pushes step progress to Node.js
`GET`	`/health`	❌	Liveness probe

NO JWT FOR FAST API BECAUSE APIS ARE ONLY EXPOSED TO THE NODE JS

5. MCP Tools & Server Implementation

Using `fastmcp` (Python) to expose deterministic tools. The AI agent uses an MCP client to call them.

`ai-agent/app/mcp/server.py`

```
from fastmcp import FastMCP
import subprocess, os, requests

mcp = FastMCP("pr-agent-tools")

@mcp.tool()
def get_pr_diff(pr_url: str, github_token: str) -> dict:
    """Fetch unified diff for a PR"""
    # GitHub API: GET /repos/{owner}/{repo}/pulls/{pr_number}
    return {"diff": "...", "files": [...]}

@mcp.tool()
def read_file(path: str, workspace: str) -> str:
    """Read file content from cloned workspace"""
    full = os.path.join(workspace, path)
    with open(full, "r") as f: return f.read()

@mcp.tool()
def write_file(path: str, content: str, workspace: str) -> bool:
    """Write/overwrite file in workspace"""
    full = os.path.join(workspace, path)
    os.makedirs(os.path.dirname(full), exist_ok=True)
    with open(full, "w") as f: f.write(content)
    return True

@mcp.tool()
def run_command(command: str, workspace: str, timeout: int = 120) -> dict:
    """Run shell command safely in workspace"""
    result = subprocess.run(
        command, shell=True, cwd=workspace, capture_output=True, text=True,
timeout=timeout
    )
    return {"stdout": result.stdout, "stderr": result.stderr, "exit_code": result.returncode}

@mcp.tool()
```

```
def create_pr(title: str, body: str, branch: str, base: str, github_token: str, repo: str) -> dict:
    """Create GitHub PR via API"""
    url = f"https://api.github.com/repos/{repo}/pulls"
    headers = {"Authorization": f"Bearer {github_token}", "Accept":
"application/vnd.github.v3+json"}
    res = requests.post(url, json={"title": title, "body": body, "head": branch, "base": base},
headers=headers)
    return res.json()
```

`ai-agent/app/mcp/__init__.py`

JWT Authentication (Node.js)

`main-service/src/middleware/auth.js`

```
const jwt = require('jsonwebtoken');

module.exports = (req, res, next) => {
  const authHeader = req.headers.authorization;
  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'Missing token' });
  }
  try {
    const token = authHeader.split(' ')[1];
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    res.status(403).json({ error: 'Invalid or expired token' });
  }
};
```

TOKEN

```
const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');

exports.login = async (req, res) => {
  const { email, password } = req.body;
  const user = await prisma.user.findUnique({ where: { email } });
  if (!user || !await bcrypt.compare(password, user.password)) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }
  const token = jwt.sign({ id: user.id, role: user.role }, process.env.JWT_SECRET, {
    expiresIn: process.env.JWT_EXPIRES_IN
  });
  res.json({ token, user: { id: user.id, email: user.email } });
};
```

Git/PR Workflow & Safety Guards

Never merge directly to `main`. Use this pipeline:

1. Webhook triggers → Node.js creates `ai/fix-{pr_id}-{timestamp}` branch from `main`
2. FastAPI clones repo to isolated `/tmp` workspace
3. AI applies fixes → commits to temp branch
4. Runs `npm test` / `pytest` / `lint` via MCP `run_command`
5. If ✅ all pass → FastAPI creates PR `ai/fix-* → main`
6. Node.js monitors CI status via GitHub Checks API
7. Auto-merge only if:
    - CI green
    - No conflict
    - `auto_merge_enabled` flag true in DB
    - Branch protection requires 0 approvals (or you bypass via PAT with `admin` scope)
8. If ❌ → AI retries (max 2x) or flags for human review

Safety Config (GitHub)

- Enable `Require status checks to pass before merging`
- Enable `Require branches to be up to date`
- Use `CODEOWNERS` for critical paths
- Set `max_commits` and `max_file_changes` limits in Node.js orchestrator

Real-Time Live Updates (WebSocket)

Node.js WebSocket Setup (`app.js`)

```
const { Server } = require('socket.io');
const http = require('http');
const express = require('express');

const app = express();
const server = http.createServer(app);
const io = new Server(server, { cors: { origin: "*" } });

// Map PR ID to socket rooms
```

```
const prRooms = new Map();

io.on('connection', (socket) => {
  socket.on('join:pr', (prId) => {
    socket.join(`pr:${prId}`);
  });
});

// Broadcast helper
exports.broadcastStep = (prId, step, status, details) => {
  io.to(`pr:${prId}`).emit('pr:step', { step, status, details, timestamp: Date.now() });
};

module.exports = { server, io, broadcastStep };
```

React Hook (`useWebSocket.ts`)

```
import { useEffect, useState } from 'react';
import { io, Socket } from 'socket.io-client';

export const usePRWebSocket = (prId: string) => {
  const [steps, setSteps] = useState<any[]>([]);
  const [socket, setSocket] = useState<Socket | null>(null);

  useEffect(() => {
    const s = io(import.meta.env.VITE_WS_URL, {
      query: { token: localStorage.getItem('token') }
    });
    s.emit('join:pr', prId);
    s.on('pr:step', (data) => setSteps(prev => [...prev, data]));
    setSocket(s);
    return () => s.disconnect();
  }, [prId]);

  return { steps, socket };
};
```

FastAPI pushes steps to Node.js via `POST /agent/webhook` with `prId, step, status, details`. Node.js calls `broadcastStep()`.

Step-by-Step Implementation Flow

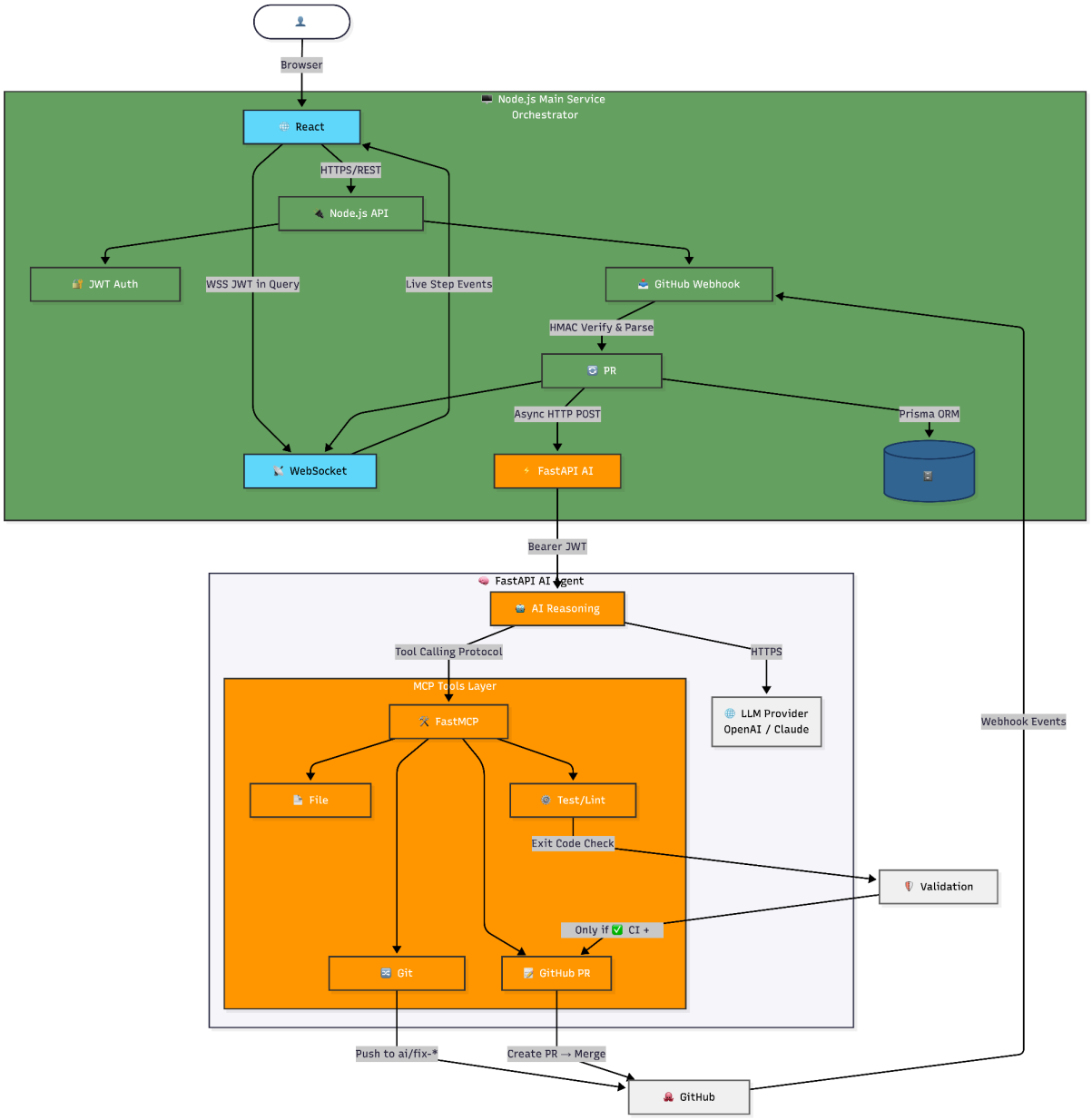
1. Setup Infra
   - Create 3 repos, initialize Node.js, FastAPI, React
   - Add `.env` files, install dependencies
   - Setup PostgreSQL + Prisma (`prisma init`, define `PRRun`, `User`, `StepLog` models)
2. Implement Auth
   - Node.js: `/api/auth/register`, `/login`, JWT middleware
   - Test with Postman
3. GitHub Webhook Receiver
   - Node.js: Verify HMAC, parse `pull_request.opened/synchronize`
   - Validate repo, PR state, create DB record
   - Spawn async job → call FastAPI `/agent/analyze`
4. MCP Server & Tools
   - FastAPI: Implement `get_pr_diff`, `read_file`, `write_file`, `run_command`, `create_pr`
   - Test tools locally with `curl` or MCP inspector
5. AI Agent Logic
   - FastAPI: Use OpenAI/Claude SDK with tool-calling
   - Prompt: "Analyze diff, suggest fixes, apply safely, run tests, report"
   - Loop until tests pass or max retries reached
6. Orchestration & Safety
   - Node.js: Create temp branch, call FastAPI, monitor status
   - Validate CI via GitHub API before merge
   - Use `octokit` npm package for GitHub operations
7. Real-Time UI
   - Node.js WebSocket setup
   - React dashboard with step timeline, logs, PR link, status badges
   - Connect via `usePRWebSocket`
8. Testing & Hardening
   - Add rate limiting, retry queues (BullMQ/Redis)
   - Isolate workspaces (`WORK_DIR`)
   - Add timeout guards to `run_command`
   - Log everything to `StepLog` table

10. Deployment & Production Notes

Component	Docker Base	Command	Notes
Node.js	`node:20-alpine`	`node src/server.js`	Expose 3000, mount secrets, use PM2 in prod
FastAPI	`python:3.11-slim`	`uvicorn app.main:app --host 0.0.0.0 --port 8000`	Add `gunicorn[uvicorn]` for workers
React	`nginx:alpine`	Serve static `dist/`	Reverse proxy via nginx

- ✅ Rotate `GITHUB_TOKEN` with least privileges (`repo`, `write:packages`, `admin:repo_hook`)
- ✅ Validate webhook signatures (`crypto.createHmac`)
- ✅ Sanitize AI-generated file paths (prevent `../../../`)
- ✅ Limit AI execution time & CPU/memory in containers
- ✅ Never store tokens in frontend or logs
- ✅ Use GitHub Branch Protection + `required_status_checks`

IF everything goes right we will use circle ci


Browser
Node.js Main Service
Orchestrator
React
HTTPS/REST
Node.js API
JWT Auth
WSS JWT in Query
Live Step Events
GitHub Webhook
HMAC Verify & Parse
PR
Async HTTP POST
Prisma ORM
WebSocket
FastAPI AI
Bearer JWT
FastAPI AI Agent
AI Reasoning
Tool Calling Protocol
HTTPS
MCP Tools Layer
FastMCP
LLM Provider
OpenAI / Claude
Webhook Events
File
Test/Lint
Exit Code Check
Validation
Only if ✅ CI +
Git
GitHub PR
Push to ai/fix-*
Create PR → Merge
GitHub